Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 5

What this is . . . 5

The C2M fault domains . . . 5

The layered root-cause model . . . 6

Named diagnostic artifacts this solution defines . . . 6

What this is not . . . 6

The deployment model . . . 7

2. Requirements Guide — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 7

2.1 A reachable C2M environment and product user . . . 7

2.2 Log and output access (WebLogic + OUAF + product) . . . 7

2.3 WLST access to the domain (offline and connected) . . . 7

2.4 Database read access for the diagnostic queries . . . 8

2.5 JMX / thread-diagnostic access . . . 8

2.6 Keystore / SSL inspection access . . . 8

2.7 A baseline of "healthy" (what normal looks like) . . . 8

2.8 A non-production environment (or captured snapshot) and change control . . . 9

Checklist . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 9

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 10

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 11

6. Collaborating teams & support model . . . 11

3. Diagnostic Playbook — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 11

Step 0 — Scope the symptom first (before any check) . . . 12

Branch A — C2M won't start / can't log in . . . 12

Branch B — Batch fails or won't submit . . . 13

Branch C — JMS backs up / messages pending . . . 14

Branch D — IWS / integration errors . . . 14

Branch E — Performance stall (slow, not down) . . . 15
Branch F — Outage (total: no tier responding) . . . 16
Step N — Restore, validate, and record the RCA . . . 16
5. Testing Guide — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 16
Test 1 — Discovery collects a clean snapshot . . . 17
Test 2 — Validation reads every layer's state . . . 17
Test 3 — Report returns the database diagnostic sections . . . 17
Test 4 — A symptom routes to the right branch . . . 17
Test 5 — The layer-isolation ladder isolates a seeded fault (non-production) . . . 17
Test 6 — Remediation previews only (dry-run safety) . . . 17
Test 7 — A restore validates clean . . . 17
Pass/fail gate . . . 18
6. Troubleshooting Guide — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 18
A. Discovery can't find the domain, logs, or utilities . . . 18
B. WLST validation cannot connect . . . 18
C. Report returns nothing or errors . . . 19
D. Thread dump / JMX not available . . . 19
E. The method points at the wrong layer (the classic traps) . . . 19
F. Remediation preview vs apply . . . 20
G. Diagnostics quick reference . . . 20
H. When to escalate to Oracle . . . 20
7. Best Practices — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 20
8. Deliverables Summary — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit 21
Documentation . . . 22
Named diagnostic artifacts . . . 22
Automation scripts (WLST/Jython, shell & SQL — not PowerShell; all read-only, remediation dry-run by default) . . . 22
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

When Oracle Utilities Customer to Meter (C2M) breaks, the cost is not the fix — it is the **hours spent guessing which layer failed**. The same symptom (a login that hangs, a screen that crawls, an integration that errors) can be a WebLogic data source that cannot reach the database, an expired keystore, a saturated thread pool, a blocking database session, or a genuine application fault. Triage by guesswork restores service late, produces no root cause, and lets the same outage return. This toolkit replaces that with a **structured root-cause tree and diagnostic scripts** for C2M and WebLogic 14c — so you isolate the failing layer, restore service, and produce a defensible root-cause analysis.

This is a complete, do-it-today package for an experienced C2M / WebLogic administrator. The method is built on four **fault domains** — WebLogic/middleware, database, network, and application/OUAF — and a single **layered ladder** (Reachable → Up → Authenticated → Resourced → Correct) walked from the cheapest, broadest layer down. Every symptom has a branch; every branch names the exact check and the layer it isolates. Every artifact is named once — the fault-domain map, the symptom decision tree, the layer-isolation checklist, the diagnostic snapshot, the root-cause record — and reused on every incident.

What you get

• A plain-language Solution Overview of the C2M fault domains and the layered root-cause model — why a

middleware symptom is so often a database or network cause, and how the ladder proves it.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — log access, WLST

connectivity, DB read access, thread-dump/JMX access, keystore inspection, and a healthy baseline — so you are ready *before* the outage.

• A Diagnostic Playbook: a symptom-driven decision tree — won't start / can't log in; batch fails; JMS

backs up; IWS/integration errors; performance stall; outage — with the exact WLST, console, SQL, and log checks per branch and the layer each isolates.

• Five automation scripts (shell, WLST/Jython, and SQL — not PowerShell): a read-only diagnostic

snapshot collector, per-layer validation, a consolidated database diagnostics report, a safe restore previewer (dry-run by default), and a scope/branch helper — authentic to C2M and WebLogic administration.

• Testing and Troubleshooting guides with explicit pass/fail gates (toolkit-ready, method-proven,

restore-complete), plus Best Practices for defensible RCA and escalation to Oracle.

What this delivers

The failing layer isolated fast, service restored with the smallest safe action, and a defensible root-cause analysis produced — instead of guesswork triage, a late restore, and a repeat outage.

The deployment model

Assess (scope the symptom, capture the snapshot), then Configure the checks, then Pilot on non-production or a captured snapshot, then Validate the restore against a gate, then Expand — with a documented rollback for every production change.

Outcome

C2M incidents resolved the repeatable way: the right layer found the first time, service restored safely, and a root-cause record that stops the repeat — with read-only scripts and a playbook the team keeps and reuses.

1. Solution Overview — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

What this is

When Oracle Utilities Customer to Meter (C2M) breaks, the hardest part is not fixing it — it is finding **which layer is actually failing**. A login that hangs can be a WebLogic data source that cannot reach the database, an expired keystore, a saturated connection pool, or a stuck thread deep in application code — and the symptom looks the same from the front. This solution is a **structured root-cause tree plus diagnostic scripts** for C2M and its Oracle WebLogic Server 14c middleware. It turns guesswork triage into a layered method: you start at the symptom, run the exact checks that isolate the failing layer, restore service, and capture a **defensible root-cause analysis (RCA)**.

> ***Version boundary.*** *Command names, script names, log paths, and object terminology below are from the Oracle Utilities Application Framework (OUAF) Server Administration Guide, the C2M product documentation, and the Oracle WebLogic Server 14c administration documentation. Version-specific details (C2M 25.4 / minimum v2.9, WebLogic 14c, OUAF batch/IWS surfaces) are* ***cited or flagged "verify for your version"*** *— never invented. Some notes and patches are gated on My Oracle Support (MOS) and are pointed to, not reproduced. See* `global-requirements-oracle.md`.

The C2M fault domains

Every C2M incident lives in one (or the seam between two) of four layers. The whole method is built on naming the layer first, because the fix — and who owns it — is different in each:

1. **WebLogic / middleware** — the Administration Server, the Managed Servers (`SPLService`, `SPLWeb`, `ohelp`), Node Manager, JVM heap and garbage collection, the execute-thread pool (stuck and hogging threads), JDBC data sources and connection pools, JMS servers/queues, and the domain's SSL/keystore configuration. Most "won't start", "can't log in", "hangs under load", and JMS-backlog symptoms surface here first.
1. **Database** — the Oracle Database schema behind C2M: listener and service availability, the accounts the data sources use, locking/blocking sessions, tablespace and temp pressure, and query plans. A middleware symptom (a data source that will not connect, a stuck thread in JDBC) is very often a database cause.
1. **Network / infrastructure** — DNS, host reachability, the ports between the tiers (Managed Server

↔ DB listener, Node Manager, JMS/RMI, the load balancer/reverse proxy to SPLWeb), firewalls, and time skew. A layer that "cannot reach" another is a network cause wearing a middleware mask.

1. **Application / OUAF** — the framework and C2M configuration and code: Inbound Web Services (IWS,

the successor to the deprecated XAI servlet), batch controls and the thread-pool workers, the MPL / messaging configuration, feature configuration, algorithms, business objects/services, and the product logs (`spl*.log`, `system.log`). When the platform is healthy but one function fails, the cause is here.

The layered root-cause model

The toolkit's core is a **top-down isolation** method — check the cheapest, broadest layer first and descend only when it passes, so you never tune the application for what is really an expired certificate:

> ***Reachable?*** → ***Up?*** → ***Authenticated?*** → ***Resourced?*** → ***Correct?*** *1.* ***Reachable*** *(network) — can the tier be reached at all (host, port, DNS, listener)? 2.* ***Up*** *(middleware) — are the Admin/Managed Servers and Node Manager* `RUNNING`*, not* `ADMIN`*,* `FAILED`*, or* `FAILED_NOT_RESTARTABLE`*? 3.* ***Authenticated*** *(middleware ↔ DB, SSL) — do the data sources connect, are the keystores/certs valid and unexpired, does login succeed? 4.* ***Resourced*** *(middleware + DB) — heap/GC, thread pool (stuck/hogging), connection pools, JMS depth, DB locks/tablespace — is anything saturated? 5.* ***Correct*** *(application/OUAF) — with the platform healthy, is the specific function (batch, IWS, a screen) configured and coded correctly?*

Each symptom branch in the Diagnostic Playbook (document 3) walks this ladder with the exact WLST, console, SQL, and log check for that branch, and states the layer each check isolates.

Named diagnostic artifacts this solution defines

So the method is repeatable and the RCA is defensible, the toolkit names its artifacts once and reuses them across every incident:

- `C2M RCA - Fault Domain Map` — the four-layer model above, the reference every triage starts from.
- `C2M RCA - Symptom Decision Tree` — the symptom-to-branch tree (won't start / can't log in;

batch fails; JMS backs up; IWS/integration errors; performance stall; outage).

- `C2M RCA - Layer Isolation Checklist` — the per-layer checks (Reachable/Up/Authenticated/

Resourced/Correct) the validation script runs.

- `C2M RCA - Diagnostic Snapshot` — the point-in-time evidence bundle (config, server states,

thread/heap, DB signals, log tails) the discovery script collects.

- `C2M RCA - Root-Cause Record` — the defensible write-up: symptom, timeline, isolating evidence,

the layer and cause, the fix applied, and prevention, produced from the report script's output.

What this is not

This is a **diagnostic** toolkit. Every included script is **read-only** where it touches a live system — discovery, validation, and report only observe (they read config, server states, JMX, and `SELECT` from the schema).

The remediation script is **dry-run by default**: it previews only safe, reversible service-restoration actions (an orderly restart, a data-source or JMS re-sync) and never changes anything unless explicitly asked, in a change window. The toolkit does not rewrite C2M business logic, patch the product, or replace the database; where a fix is a product defect it points to the version's MOS note and a service request rather than reproducing gated content.

The deployment model

> *Reproduce and scope the symptom, run the layer-isolation checks to find the failing layer, restore service with the smallest safe action, validate the restore against a gate, then record the RCA and the prevention. The repeatable way: Assess, then Configure (the checks), then Pilot (on non-production or a captured snapshot), then Validate, then Expand — with a documented rollback for every production change.*

2. Requirements Guide — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

Verify every requirement before you rely on the toolkit in an incident — the time to confirm you have log access and WLST connectivity is *before* the outage, not during it. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (licensing, certified stack, environments, WLST/shell/SQL tooling, change control) are in `global-requirements-oracle.md`.

2.1 A reachable C2M environment and product user

Description: A licensed C2M environment with its OUAF product home reachable, and the OS product user that owns it (the account under which the WebLogic domain, `threadpoolworker`, and IWS run), with the environment sourced via `splenviron.sh -e your-env`. Why: The diagnostic scripts locate the domain, logs, and utilities through the variables `splenviron.sh` sets (`SPLEBASE`, `SPLOUTPUT`); without them they cannot find what to inspect. Verify: As the product user, run `splenviron.sh -e your-env` then run `OC2MRCA-Discovery.sh` — it confirms the environment is sourced and the domain/log paths resolve. Implement-if-missing: Obtain the product-user credentials and the environment name; confirm the OUAF install is complete (`initialSetup`/`configureEnv` run) before proceeding.

2.2 Log and output access (WebLogic + OUAF + product)

Description: Read access to the log tree: the WebLogic domain logs and per-server `.out`/`.log` files (including the Admin and Managed Server logs and any Node Manager log), and the OUAF/product logs under `SPLOUTPUT`/the product log directory (`spl*.log`, `system.log`, batch and IWS logs). Why: The log is the primary evidence for every branch of the tree — a `kill -3` thread dump lands in a server `.out`, a data-source failure and a startup failure both write a diagnosable stack, and the RCA is built from these tails. No log access means no defensible RCA. Verify: You can read the current Managed Server `.out` and the product `spl*.log`; run `OC2MRCA-Discovery.sh`, which tails and fingerprints the standard logs (read-only). Implement-if-missing: Grant read on the domain and product log directories to the diagnostic account (or run as the product user); confirm `SPLOUTPUT` and the WebLogic domain log path.

2.3 WLST access to the domain (offline and connected)

Description: WebLogic Scripting Tool (WLST, Jython) available and able to connect to the running domain (to the Admin Server), plus the domain admin URL and credentials, so server states, thread pools, JDBC, and JMS runtime can be read. Why: WLST is how the toolkit reads live middleware state — server health (`RUNNING`/`ADMIN`/`FAILED`), the execute-thread pool (stuck/hogging counts, health state), data-source connection-pool state, and JMS destination depth — without clicking through the console during an incident. Verify: `java weblogic.WLST` starts, and `connect()` to the Admin URL succeeds; run `OC2MRCA-Validation.py` under WLST, which reports server and resource state. Implement-if-missing: Install/point to the WLST from the domain's `oracle_common/wlserver`, and obtain the Admin Server URL (`t3://...`) and read-capable admin credentials.

2.4 Database read access for the diagnostic queries

Description: A read (SELECT) account on the C2M schema and the ability to reach the database as the data sources do, so the toolkit can check listener/service availability, blocking sessions, tablespace and temp pressure, and batch/IWS run status from the database side. Why: A large share of "middleware" symptoms (a data source that will not connect, a stuck JDBC thread, a slow screen) are database causes; the layered model descends into the DB, and that needs SQL access. Verify: You can connect with SQL*Plus/SQLcl and select from the schema; run `OC2MRCA-Report.sql` (read-only) and confirm it returns the diagnostic sections. Implement-if-missing: Obtain a read-only DB account (or use the product user's), and confirm SQL connectivity and the service name per the global requirements.

2.5 JMX / thread-diagnostic access

Description: The ability to obtain a JVM thread dump for each Managed Server (via the console's Monitoring → Threads *Dump Thread Stacks*, `jstack`, or `kill -3` to the server `.out`), and — for batch — the OUAF batch JMX interface (`jmxbatchclient.sh`, RMI/JMX port populated, e.g. `BATCH_RMI_PORT`) for live batch threads. Why: Stuck and hogging threads are diagnosed from thread dumps (the top of the stack names the layer — JDBC = database, a lock = deadlock, product code = application); JMX is how live batch threads are seen without bouncing a worker. Verify: You can produce a thread dump for a Managed Server and it lands in the expected `.out`; `jmxbatchclient.sh -s` returns a summary. `OC2MRCA-Validation.py` reports the thread-pool health state. Implement-if-missing: Confirm `kill -3`/`jstack` access to the server processes and that the batch RMI/JMX port is set; grant the OS access needed to signal the JVMs.

2.6 Keystore / SSL inspection access

Description: Access to inspect (not necessarily change) the domain's identity/trust keystores and the certificates the SSL listeners and outbound integrations use — the keystore files/paths and `keytool`/`orapki` to read expiry and chains. Why: An expired or mis-trusted certificate presents as "can't log in", a failed outbound IWS call, or a Managed Server that will not reach a secured resource — a distinct layer that the tree checks explicitly so it is not misdiagnosed as an application fault. Verify: You can list certificate expiry from the identity/trust store with `keytool -list`; `OC2MRCA-Validation.py` flags certificates near or past expiry (read-only). Implement-if-missing: Obtain read access to the keystore files and the store passwords needed only to *list* contents; document the keystore paths from the domain SSL configuration.

2.7 A baseline of "healthy" (what normal looks like)

Description: A recorded baseline for the environment — normal startup log signature, typical heap/GC and thread-pool levels, normal batch durations, normal JMS depth, and expected server states — so an incident's snapshot can be compared to normal rather than to a guess. Why: RCA is a comparison: "this is what changed." Without a baseline every reading is ambiguous; with one, the deviating layer stands out immediately. Verify: A baseline snapshot exists (capture one now with `OC2MRCA-Discovery.sh` on a healthy day); `OC2MRCA-Report.sql` and the discovery output give you the comparison set. Implement-if-missing: Run the discovery and report scripts on a known-good day and store the output as the baseline `C2M RCA - Diagnostic Snapshot`.

2.8 A non-production environment (or captured snapshot) and change control

Description: A non-production C2M environment to reproduce symptoms and rehearse the toolkit safely, or — when the fault is production-only — the discipline to capture a read-only `C2M RCA - Diagnostic Snapshot` and test fixes on non-production, plus change control and backups (domain config, product config) before any restart or re-sync in production. Why: Even "safe" restorative actions (an orderly restart, a data-source re-sync) are production events; the model is Assess → Configure → Pilot → Validate → Expand with a documented rollback. Verify: A non-production environment is available and a backup/rollback path for the domain and product config is confirmed. Implement-if-missing: Stand up or obtain the non-production environment and establish the backup and change window before any production restore action.

Checklist

- C2M environment reachable, product user, `splenviron.sh -e your-env` sources cleanly — `OC2MRCA-Discovery.sh`
- Log/output access (WebLogic `.out`/domain logs + OUAF `spl*.log`/`system.log`) — `OC2MRCA-Discovery.sh`
- WLST connects to the Admin Server (read server/resource state) — `OC2MRCA-Validation.py`
- DB read access for the diagnostic queries — `OC2MRCA-Report.sql`
- Thread-dump + batch JMX access (stuck/hogging + live batch threads) — `OC2MRCA-Validation.py`, `jmxbatchclient.sh`
- Keystore/SSL inspection access (cert expiry) — `OC2MRCA-Validation.py`
- A recorded "healthy" baseline snapshot — `OC2MRCA-Discovery.sh`, `OC2MRCA-Report.sql`
- Non-production (or captured snapshot) + change control and backups

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Diagnostic Playbook — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

This is the working core of the toolkit: a **symptom-driven decision tree**. Find your symptom, walk its branch top-down through the layer-isolation ladder, and each check tells you the exact WLST / console /

SQL / log command to run and the **layer it isolates**. Do not skip down the ladder — a check that passes rules out a layer; a check that fails is your root cause (or the next layer to descend into). Capture evidence as you go with `OC2MRCA-Discovery.sh` so the RCA writes itself.

> *Model: reproduce and scope the symptom, isolate the failing layer with the checks below, restore service with the smallest safe action, validate the restore against the gate, then record the RCA. Assess, then Configure the checks, then Pilot (on non-production or a captured snapshot), then Validate, then Expand — with a documented rollback for every production change.*
>
> *All commands assume the environment is sourced first:* `splenviron.sh -e your-env` *(run as the product user). WLST connects to the Admin Server (*`connect()` *to the* `t3://` *admin URL). Command, script, log, and object names are from the OUAF Server Administration Guide, the C2M documentation, and the WebLogic 14c administration documentation —* ***verify exact names/flags/paths for your version****. MOS-gated notes are pointed to, not reproduced.*

Step 0 — Scope the symptom first (before any check)

1. Run `OC2MRCA-ScopeHelper.sh --symptom your-symptom` to bound the incident: which tier(s), which

Managed Server(s)/node(s), when it started, what changed (deploy, patch, cert, data spike), and whether it is total (outage) or partial (one function). This selects the branch below and the target set the other scripts run against.

1. Capture a starting `C2M RCA - Diagnostic Snapshot` with `OC2MRCA-Discovery.sh` (read-only): server

states, log tails, thread/heap, DB signals. This is timestamped evidence for the RCA and the "before" for any restore.

Branch A — C2M won't start / can't log in

The two most common openers, walked down the same ladder.

A. Reachable? (network layer)

- **Check:** From the app host, confirm the DB listener/service and the tiers answer:

`tnsping your-service`, and a port check to the DB listener and to the Managed Server ports. DNS resolves the DB and LB hostnames.

- **Isolates:** Network. If a required port/host is unreachable, the cause is network/infrastructure —

fix routing/DNS/firewall/listener before touching WebLogic. If reachable, descend.

B. Up? (middleware — server state)

- **Check (won't start):** Is **Node Manager** running and are the servers actually being started?

Tail the server `.out/.log` under the domain log dir; under WLST `connect()` then `state('SPLService')` / `state('SPLWeb')`. A common cause after an unclean shutdown is orphaned Java processes holding the ports — confirm no stale JVM is bound before restart.

- **Check (health state):** `FAILED`, `FAILED_NOT_RESTARTABLE`, or stuck in `ADMIN` mode each point

somewhere specific — `ADMIN` mode after a domain change often means a JRF/classpath problem, and a `ClassNotFoundException` in the `.out` names it (see the MOS note for JRF-in-admin-mode).

- **Isolates:** Middleware startup. `OC2MRCA-Validation.py` reports each server's state and health.

C. Authenticated? (middleware ↔ DB, and SSL)

- **Check (can't log in / data source):** Under WLST read the JDBC runtime — does the C2M data source

connect, or is it disabled/suspended with connection failures? A login that hangs or fails with the platform "up" is very often the data source unable to reach the DB or the DB account locked/expired. Confirm the DB side with `OC2MRCA-Report.sql` (blocking, account status).

- **Check (SSL):** List the identity/trust keystore expiry (`keytool -list`); an expired listener or

trust cert presents as a failed login or a Managed Server that will not reach a secured resource. `OC2MRCA-Validation.py` flags near/expired certs.

- **Isolates:** DB connectivity/credentials, or SSL/keystore. Fixing here restores login without any

application change.

D. Resourced? (middleware + DB)

- **Check:** Heap/GC not exhausted (repeated full GC / `OutOfMemoryError` in the `.out`), the

execute-thread pool not saturated with stuck threads (health `warning`/`critical`), the connection pool not at max with pending requests. `OC2MRCA-Validation.py` reports thread-pool health and pool usage.

- **Isolates:** Resource saturation at startup/login. If starved, restore capacity (see Remediation)

before blaming config.

E. Correct? (application/OUAF)

- **Check:** With platform healthy, is the product itself configured to start — `configureEnv`/

`initialSetup` applied, the EARs (`SPLService`, `SPLWeb`, `ohelp`) deployed, feature configuration valid? Product startup errors land in `spl*.log`/`system.log`.

- **Isolates:** Application/OUAF configuration. This is the layer only after network, server, auth,

and resources pass.

Branch B — Batch fails or won't submit

A. Up? (worker + daemon)

- **Check:** Are the `threadpoolworker` JVMs running and advertising their pools, and is the batch

daemon node polling submissions? `OC2MRCA-Validation.py` / `jmxbatchclient.sh -s` for live threads; `OC2MRCA-Discovery.sh` for worker config. A control that "never runs" with no daemon is a worker/ daemon-up problem, not a batch-code one.

- **Isolates:** Middleware (batch runtime).

B. Authenticated / Resourced? (DB + pool)

• **Check:** Batch runs largely in JDBC — a stuck batch thread whose dump tops out in JDBC is a database cause: blocking session, tablespace/temp full, a bad plan. `OC2MRCA-Report.sql` shows batch run status, in-flight/overdue threads, blocking sessions, and tablespace pressure.

• **Isolates:** Database (the usual batch culprit) or connection-pool starvation.

C. Correct? (batch control / data)

• **Check:** With runtime and DB healthy, the control itself errors — read the batch log and the run status (`batch_thread_stat_flg` error rows via `OC2MRCA-Report.sql`); a repeatable error on one control with clean infrastructure is a control config/data issue for the application team.

• **Isolates:** Application/OUAF. If it is a product defect, point to the version's MOS note / open an SR.

Branch C — JMS backs up / messages pending

A. Up? (consumers attached)

• **Check:** Open the JMS destination runtime (WLST or console Monitoring): are **consumers present** (consumer count > 0)? A backlog with **zero consumers** means the consuming component (an OUAF IWS/MPL consumer, an MDB) is down or not attached — that is the cause, not the queue.

• **Isolates:** Middleware (consumer availability).

B. Resourced / draining? (throughput)

• **Check:** Consumers attached but **pending count climbs** — messages arrive faster than they drain, or messages sit in *send/pending-transaction* state (a transaction resource in the global transaction is stuck). Check the consumer's own errors (product log) and any downstream (DB/outbound) it calls; `OC2MRCA-Report.sql` correlates with DB blocking.

• **Isolates:** Consumer throughput / a stuck transaction / a slow downstream — descend to whichever the consumer's stack names.

C. Correct? (message / config)

• **Check:** Messages move to an error/dead queue, or the consumer rejects them — a malformed message or a mis-mapped service. Read the consumer's product log; this is application/OUAF configuration.

• **Isolates:** Application/OUAF. Redelivery/error-queue settings are config, not a queue defect.

Branch D — IWS / integration errors

(Inbound Web Services — the successor to the deprecated XAI servlet; XAI-via-IWS support is a Feature Configuration option.)

A. Reachable / Up?

• **Check:** Does the endpoint answer at all — is IWS **enabled** (it is disabled by default for backward compatibility and must be turned on), is the service **deployed** (not stuck in "Deployment in Progress"/`ERROR`), and does the URL resolve through the LB/reverse proxy to `SPLWeb`?

• **Isolates:** Network reachability or middleware deployment state.

B. Authenticated? (security / SSL)

• **Check:** A `401`/`403` or an SSL handshake failure — the caller's credentials, the security policy on the service, or an expired/untrusted certificate on an outbound call. `OC2MRCA-Validation.py` flags cert expiry; the product/HTTP log shows the auth failure.

• **Isolates:** Auth/SSL — a distinct layer from the service logic.

C. Correct? (service mapping / payload)

• **Check:** A `500`/business fault with the platform healthy — the service references the wrong business object/service/script, or the payload/schema is wrong. Read the IWS/product log for the fault; for an XAI-via-IWS service confirm the External Messages *Support XAI Services via IWS* feature option is set.

• **Isolates:** Application/OUAF service configuration.

Branch E — Performance stall (slow, not down)

A. Resourced — JVM? (middleware)

• **Check:** Thread pool full of **stuck/hogging** threads (default: continuously busy > 600s = stuck; all default-queue threads stuck → health `critical`) and/or heap in constant full GC. Take a thread dump (`kill -3` → server `.out`, or console *Dump Thread Stacks*) and read the **top of the stack**: JDBC = database, an object-lock wait = deadlock, product code = application. `OC2MRCA-Validation.py` reports stuck/hogging counts and health state.

• **Isolates:** Points you at the next layer by what the stuck threads are waiting on.

B. Resourced — Database? (DB)

• **Check:** If dumps top out in JDBC — blocking sessions, a plan regression, tablespace/temp pressure, or connection-pool exhaustion. `OC2MRCA-Report.sql` lists blocking sessions, long-running SQL signals, tablespace/temp usage, and pool pressure.

• **Isolates:** Database. Most "everything is slow" with clean JVM CPU is a DB cause.

C. Correct? (application/OUAF)

• **Check:** Platform and DB healthy but one function is slow — a specific algorithm, batch control, or IWS path. Read that function's product log and its SQL. Compare to the healthy **baseline** (Requirement 2.7) — what deviates is the cause.

• **Isolates:** Application/OUAF.

Branch F — Outage (total: no tier responding)

Fastest path from "everything is down" to the failed layer.

1. **Reachable (network):** Is it the whole host/network or C2M? Ping/port-check the DB listener, the

Managed Server ports, and the LB. A dead listener or a network partition is the outage — restore it first. (`OC2MRCA-ScopeHelper.sh --symptom outage` bounds which tiers are dark.)

1. **Up (middleware):** If reachable, are the servers down? WLST `state()` / server `.out` — a crashed

Admin/Managed Server (OOM, disk full on the log/output volume, an unclean shutdown holding ports). Check disk on the domain and `SPLOUTPUT` volumes early; a full log volume takes C2M down and is easy to miss.

1. **Authenticated (DB):** If servers are up but every request fails, the database or its listener is

down / the data source cannot connect — confirm with `OC2MRCA-Report.sql`/`tnsping`. A DB outage presents as a total C2M outage.

1. **Restore, smallest action first:** Restore the failed layer with the least, most reversible action

(start the listener, free the disk, orderly-restart the failed server) — preview it with `OC2MRCA-Remediation.sh` (dry-run) — then re-validate. Capture the snapshot **before** restart so the RCA is not lost when logs roll.

Step N — Restore, validate, and record the RCA

1. **Restore** with the smallest safe action. `OC2MRCA-Remediation.sh` is **dry-run by default** — it

previews the exact restart / data-source re-sync / JMS re-sync it *would* do and changes nothing; apply only with the explicit flag in a change window, backups taken.

1. **Validate the restore** against the gate (Testing Guide): the failed layer's check now passes,

dependent layers pass, and the symptom is gone under a representative load.

1. **Record** `C2M RCA - Root-Cause Record` from the collected evidence: symptom + scope, timeline,

the isolating check that named the layer, the root cause, the fix applied, and the prevention. `OC2MRCA-Report.sql` and the discovery snapshots supply the evidence; escalate genuine product defects to Oracle with the snapshot attached (see Best Practices).

Order of operations

```
Scope symptom -> Reachable? (net) -> Up? (middleware) -> Authenticated?
(DB/SSL) -> Resourced? (JVM/DB/pools/JMS) -> Correct? (app/OUAF) -> Restore
(smallest safe action) -> Validate gate -> Record RCA
```

5. Testing Guide — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

Two things get tested here: first that the **toolkit itself works** in your environment (run these on a healthy day so the scripts are proven before an incident), and second that a **restore is complete** (run the relevant checks after any fix). Each test confirms one layer-isolation check returns the expected reading. Source the environment first: `splenviron.sh -e your-env`; WLST connects to the Admin Server.

Test 1 — Discovery collects a clean snapshot

Do: Run `OC2MRCA-Discovery.sh` on a healthy environment. Expected: It reports environment sourced OK, resolves the domain and product log paths, tails the standard logs, and emits a `C2M RCA - Diagnostic Snapshot` — with no error. Store it as the baseline.

Test 2 — Validation reads every layer's state

Do: Run `OC2MRCA-Validation.py` under WLST (`java weblogic.WLST OC2MRCA-Validation.py`). Expected: For each layer it prints a reading: server states (`RUNNING`), thread-pool health (`ok`, stuck/hogging counts), JDBC data-source/pool state, JMS destination consumer/pending counts, and certificate expiry. On a healthy environment every layer reads PASS.

Test 3 — Report returns the database diagnostic sections

Do: Run `OC2MRCA-Report.sql` via SQL*Plus/SQLcl against the schema. Expected: All sections return — batch run status, in-flight/overdue threads, blocking sessions, tablespace/temp pressure, and any error rows. On a healthy day: no blocking, no overdue thread, no error rows, tablespace within threshold.

Test 4 — A symptom routes to the right branch

Do: For a known past incident (or a reproduced one on non-production), run `OC2MRCA-ScopeHelper.sh --symptom your-symptom`. Expected: It bounds the incident to the correct tier/servers and selects the matching Diagnostic Playbook branch and target set — the tree points at the layer that actually failed.

Test 5 — The layer-isolation ladder isolates a seeded fault (non-production)

Do: On non-production, seed one fault per layer in turn (stop the listener = network/DB; suspend a data source = auth; drive threads to stuck = resource; misconfigure one IWS service = application) and walk the branch. Expected: Each seeded fault is caught at the correct rung — the ladder stops descending at the failing layer and names it, without false-blaming a lower layer.

Test 6 — Remediation previews only (dry-run safety)

Do: Run `OC2MRCA-Remediation.sh` with no apply flag. Expected: It prints the exact restore action it *would* take (orderly restart / data-source re-sync / JMS re-sync) and **changes nothing** — exit without side effects. It never restarts a live server by itself.

Test 7 — A restore validates clean

Do: After applying a fix (non-production, or production in a change window), re-run the failed layer's check (`OC2MRCA-Validation.py` / `OC2MRCA-Report.sql`) and exercise the symptom under representative load. Expected: The previously failing layer now reads PASS, dependent layers still PASS, and the

symptom does not reproduce. This is the restore gate.

Pass/fail gate

Gate	Requirement
Toolkit ready	Tests 1-4 pass on a healthy environment: discovery snapshots, validation reads every layer, report returns all sections, scope routes correctly — and a baseline is stored
Method proven	Test 5 passes: each seeded per-layer fault is isolated at the correct rung on non-production
Restore complete	Tests 6-7 pass: remediation previews safely, and after a fix the failed layer reads PASS with the symptom gone under load, dependent layers still PASS

6. Troubleshooting Guide — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

This guide troubleshoots **the toolkit and the method** — the failure modes you hit while running the diagnostics themselves, and the traps that cause a wrong-layer diagnosis. (The symptom-to-fix content for C2M incidents is the Diagnostic Playbook, document 3.) Use `OC2MRCA-Discovery.sh` and `OC2MRCA-Validation.py` for state, `OC2MRCA-Report.sql` for the DB view, and `OC2MRCA-Remediation.sh` (dry-run by default) for safe restore previews. Source the environment first: `splenviron.sh -e your-env`. MOS-gated notes are pointed to, not reproduced.

A. Discovery can't find the domain, logs, or utilities

Symptom	Cause	Remediation
"environment not sourced" / empty `SPLEBASE`	`splenviron.sh` not run in this shell	Run `splenviron.sh -e your-env` as the product user, then re-run
Log paths don't resolve	`SPLOUTPUT`/domain log path wrong or no read access	Confirm the product and WebLogic domain log directories and grant read to the diagnostic account (Requirement 2.2)
Utility not found	Utility name differs by version	Verify the utility name/path in the Server Administration Guide for your version

B. WLST validation cannot connect

Symptom	Cause	Remediation
`connect()` refused/timeout	Wrong admin URL, Admin Server down, or port blocked	Confirm the `t3://` admin URL and that the Admin Server is `RUNNING`; check the port is reachable (this is itself a network-layer finding)
Auth failure	Wrong/insufficient admin credentials	Use a read-capable admin account; confirm credentials (Requirement 2.3)
WLST won't launch	Wrong WLST/classpath	Launch from the domain's `oracle_common/wlserver` WLST; source the environment first

C. Report returns nothing or errors

Symptom	Cause	Remediation
No rows / ORA on connect	Wrong service or no SELECT grant	Confirm the service name and grant read on the C2M schema (Requirement 2.4)
Table/column not found	Object names differ by version	The queries follow the OUAF/C2M model — verify object/column names for your version (they are cited, not invented)
Status codes look wrong	Status-flag values differ by version	Confirm the batch/JMS status flag values for your release before acting on them

D. Thread dump / JMX not available

• A stuck-thread branch needs a dump but you cannot get one: confirm `kill -3`/`jstack` access to the server JVMs (the dump lands in the server `.out` under the domain log dir), or use the console Monitoring → Threads *Dump Thread Stacks*. For live batch threads confirm the batch RMI/JMX port (e.g. `BATCH_RMI_PORT`) is populated and `jmxbatchclient.sh -s` connects (Requirement 2.5).

E. The method points at the wrong layer (the classic traps)

• **Skipping the ladder.** Tuning the application or the DB while an **expired certificate** or a **down listener** is the real cause. Always walk Reachable → Up → Authenticated → Resourced → Correct in order; do not jump to "Correct".

• **Reading a symptom as its layer.** A data source that "won't connect" is usually a **DB or network** cause, not a WebLogic bug; a stuck thread is named by the **top of its stack**, not by the fact that it is in WebLogic. Read the evidence, not the surface.

• **No baseline.** Without the healthy `C2M RCA - Diagnostic Snapshot` (Requirement 2.7) every reading

is ambiguous. Capture the baseline on a good day so "what changed" is answerable.

• **Losing evidence to log roll.** Restarting before capturing the snapshot destroys the RCA. Always

run `OC2MRCA-Discovery.sh` **before** any restart.

F. Remediation preview vs apply

• `OC2MRCA-Remediation.sh` with no flag is **dry-run** — it prints the intended restore and changes

nothing. It never restarts a running server itself; it prints the exact restart/re-sync step for the admin to run under change control. Only `--apply`, in a change window with backups taken, performs it. If a "safe" restore does not clear the symptom, you diagnosed the wrong layer — go back up the ladder.

G. Diagnostics quick reference

```
splenviron.sh -e your-env                          # source the environment first
./OC2MRCA-Discovery.sh                              # snapshot: server states, logs, thread/heap, DB signa
java weblogic.WLST OC2MRCA-Validation.py            # per-layer checks: servers, threads, JDBC, JMS, certs
sqlplus user@db @OC2MRCA-Report.sql                # DB view: batch/JMS status, blocking, tablespace (read
./OC2MRCA-ScopeHelper.sh --symptom performance      # bound the incident, select the branch
./OC2MRCA-Remediation.sh                            # dry-run by default; add --apply in a change window
```

H. When to escalate to Oracle

If the failing layer is isolated, the platform is healthy, and the fault persists after the safe restore — or the evidence points to a product defect (a stack in product code, a known error string) — capture the `C2M RCA - Diagnostic Snapshot` (discovery + validation + report + the thread dump) and open a service request. Consult the version's My Oracle Support notes (JRF-in-admin-mode, IWS-deployment, pending-JMS, and similar are documented there); gated content is pointed to, not reproduced here. Attach the snapshot so Oracle sees the isolating evidence, not just the symptom.

7. Best Practices — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

• Isolate the layer before you fix anything. Walk the ladder every time — Reachable (network) → Up

(middleware) → Authenticated (DB/SSL) → Resourced (JVM/DB/pools/JMS) → Correct (application/OUAF). The single biggest cause of long outages is fixing the wrong layer; a passing rung *rules a layer out*, a failing rung *is* the cause or the next descent.

• Capture the snapshot before you restart. Run `OC2MRCA-Discovery.sh` first, always — server states,

thread/heap dumps, log tails, DB signals. A restart clears the symptom *and* the evidence; without the `C2M RCA - Diagnostic Snapshot` there is no defensible RCA and the incident will recur unexplained.

• Keep a healthy baseline. RCA is a comparison — "this is what changed." Record a known-good snapshot

(`OC2MRCA-Discovery.sh` + `OC2MRCA-Report.sql`) on a normal day so the deviating layer stands out instead of every reading looking suspicious.

• Read the top of the thread dump, not the surface. A stuck thread is named by where it is waiting:

JDBC at the top = database, an object-lock wait = deadlock, product code = application. "It's stuck in WebLogic" is not a root cause; the stack is.

• Treat "won't connect" as network-or-DB until proven otherwise. A suspended data source, a login that

hangs, a failed outbound call — the cause is usually a down listener, a locked/expired DB account, a firewall, or an **expired certificate**, not a WebLogic defect. Check reachability and certs early.

• Restore with the smallest reversible action. Prefer starting the listener, freeing the disk, or an

orderly single-server restart over a full-stack bounce. Preview with `OC2MRCA-Remediation.sh` (dry-run) and apply only in a change window with backups — the least action that restores service is the safest and the easiest to roll back.

• Watch the cheap, high-signal resources first in a stall. Full GC / `OutOfMemoryError` in the `.out`,

stuck/hogging thread counts (default stuck = busy > 600s; all default-queue stuck → health `critical`), connection-pool at max, JMS pending climbing with zero consumers — these four catch most performance and backlog incidents before any deep dive.

• Prefer IWS, and know its defaults. Inbound Web Services superseded the deprecated XAI servlet; IWS is

disabled by default and must be enabled, and XAI-via-IWS needs the External Messages *Support XAI Services via IWS* feature option. A "broken integration" is often just a service not enabled or not deployed — check state before logic.

• Rehearse on non-production, or on a captured snapshot. Prove the toolkit and seed per-layer faults on

non-production so the method is muscle memory before the outage. When a fault is production-only, diagnose from the read-only snapshot and test the fix on non-production first.

• Write the Root-Cause Record every time, even for quick fixes. `C2M RCA - Root-Cause Record` —

symptom, timeline, the isolating check, the layer and cause, the fix, and the prevention — is what stops the repeat outage and what a defensible post-incident review (and Oracle) needs.

• Escalate with evidence, not symptoms. When it is a product defect, attach the full snapshot to the

service request and cite the version's My Oracle Support note — gated content is pointed to, never reproduced. Oracle resolves an isolated, evidenced fault far faster than "C2M is slow".

• Change under change control only. Every restore action in production — restart, data-source re-sync,

JMS re-sync — is a change: backups first (domain and product config), a change window, and a documented rollback. "Safe" still means controlled.

8. Deliverables Summary — Oracle C2M Troubleshooting & Root-Cause Analysis Toolkit

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M fault domains (WebLogic / database / network / application-OUAF) and the layered root-cause model (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Diagnostic Playbook	The symptom-driven decision tree: won't start / can't log in; batch fails; JMS backs up; IWS/integration errors; performance stall; outage — exact checks per branch and the layer each isolates
5. Testing Guide	Toolkit-ready and restore-complete procedures with pass/fail gates
6. Troubleshooting Guide	Failure modes of the diagnostics themselves and the wrong-layer traps
7. Best Practices	Defensible RCA, layer-isolation discipline, and escalation to Oracle

Named diagnostic artifacts

Artifact	Name
Four-layer fault model	`C2M RCA - Fault Domain Map`
Symptom-to-branch tree	`C2M RCA - Symptom Decision Tree`
Per-layer isolation checks	`C2M RCA - Layer Isolation Checklist`
Point-in-time evidence bundle	`C2M RCA - Diagnostic Snapshot`
Defensible root-cause write-up	`C2M RCA - Root-Cause Record`

Automation scripts (WLST/Jython, shell & SQL — not PowerShell; all read-only, remediation dry-run by default)

Script	What it does
`OC2MRCA-Discovery.sh`	Collects the `C2M RCA - Diagnostic Snapshot`: environment/domain/log resolution, server states, thread/heap and log tails, and DB signal pointers (read-only)

Script	What it does
`OC2MRCA-Validation.py`	WLST per-layer checks: server states/health, execute-thread pool (stuck/hogging), JDBC data-source/pool state, JMS consumer/pending counts, and certificate expiry (read-only)
`OC2MRCA-Report.sql`	Consolidated database diagnostics: batch/JMS run status, in-flight/overdue threads, blocking sessions, tablespace/temp pressure, error rows (read-only)
`OC2MRCA-Remediation.sh`	Previews the smallest safe restore (orderly restart / data-source re-sync / JMS re-sync); **dry-run by default**, applies only with an explicit flag in a change window
`OC2MRCA-ScopeHelper.sh`	Bounds the incident to tier/servers, selects the matching Diagnostic Playbook branch, and emits the target set the other scripts run against

What the buyer receives

- This Diagnostic Playbook and its guides as a branded Word document and PDF.
- All five automation scripts (shell / WLST-Jython / SQL) in a single ZIP.
- A repeatable diagnostic method following the model: Assess (scope + snapshot), then Configure (the checks), then Pilot (non-production or a captured snapshot), then Validate (restore gate), then Expand.

Outcome

The failing layer of a C2M problem isolated fast — service restored with the smallest safe action and a defensible root-cause analysis produced — with a repeatable playbook and read-only scripts the buyer keeps and reuses on every future incident.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MRCA-Discovery.sh

Purpose: Collect a point-in-time C2M RCA - Diagnostic Snapshot for an Oracle Utilities C2M environment and its WebLogic 14c domain: confirm the environment is sourced, resolve the domain and product log paths, list the WebLogic server output/log files and tail the most recent entries, capture any thread-dump / OutOfMemoryError / startup-error fingerprints from the server .out files, and point to the database diagnostics. Read-only — it changes nothing. Run it FIRST, before any restart, so the evidence is captured before logs roll (Diagnostic Playbook Step 0). Also run it on a healthy day to store the baseline. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MRCA-Discovery.sh Optional: OUT_DIR=/path/to/snapshot to write the snapshot files to a folder. Optional: DB_CONN=user/pass@your-db to include a quick DB reachability note. EXPECTED RESULTS Prints: environment sourced OK and domain/log paths resolved; the WebLogic server .out/.log inventory with recent tails; any thread-dump / OOM / FAILED markers found; the product spl*.log / system.log tails; and a pointer to run OC2MRCA-Report.sql for the DB view. PASS = environment sourced and log paths resolved. FAIL = environment not sourced or log paths missing — fix before continuing. NOTES White-label, repeatable discovery — the same snapshot every incident and for the baseline. Read-only where it reads live systems: it only reads config/log files and, if DB_CONN is set, issues a single reachability SELECT. Command/script/log names are from the OUAF Server Administration Guide and WebLogic 14c docs — verify exact names/paths for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MRCA-Discovery.sh Optional: OUT_DIR=/path/to/snapshot to write the snapshot files to a folder. Optional: DB_CONN=user/pass@your-db to include a quick DB reachability note. EXPECTED RESULTS Prints: environment sourced OK and domain/log paths resolved; the WebLogic server .out/.log inventory with recent tails; any thread-dump / OOM / FAILED markers found; the product spl*.log / system.log tails; and a pointer to run OC2MRCA-Report.sql for the DB view. PASS = environment sourced and log paths resolved. FAIL = environment not sourced or log paths missing — fix before continuing. NOTES White-label, repeatable discovery — the same snapshot every incident and for the baseline. Read-only where it reads live systems: it only reads config/log files and, if DB_CONN is set, issues a single reachability SELECT. Command/script/log names are from the OUAF Server Administration Guide and WebLogic 14c docs — verify exact names/paths for your version.

OC2MRCA-Remediation.sh

Purpose: Once the failing layer is isolated, preview the SMALLEST safe, reversible restore for it and print the exact command the admin runs under change control. DRY-RUN by default: it prints only what it WOULD do and changes nothing. It covers the safe restores only — an orderly single-server restart, a JDBC data-source re-sync (suspend/resume, i.e. a reset without config change), and a JMS re-sync (pause/resume consumption) — never a config edit, never a product change, never a full-stack bounce by itself. It does not restart a running server on its own even with --apply; it emits the precise WLST/OS step for the admin to run in the change window. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MRCA-Remediation.sh --action restart --target SPLService # DRY-RUN preview ./OC2MRCA-Remediation.sh --action datasource --target C2M-DS # DRY-RUN preview ./OC2MRCA-Remediation.sh --action jms --target C2M-Queue # DRY-RUN preview

./OC2MRCA-Remediation.sh --action restart --target SPLService --apply # change window only Actions: restart | datasource | jms Target: server / data-source / destination name. Admin URL for the emitted WLST steps: ADMIN_URL=t3://your-admin-host:7001 EXPECTED RESULTS DRY-RUN prints the isolated layer, the smallest safe restore for it, and the EXACT command(s) to apply (with the "capture snapshot + backup first" and "re-validate after" reminders) — and exits without changing anything. With --apply it prints the same steps framed as a change-window runbook to execute; it still does not perform a live restart itself. Always re-run OC2MRCA-Validation.py afterwards to confirm the restore. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. Read-only in dry-run. It only ever previews reversible, service-restoration actions; anything that edits config or changes the product is out of scope by design. Command names/flags follow the WebLogic 14c / OUAF model — verify for your version before use.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MRCA-Remediation.sh --action restart --target SPLService # DRY-RUN preview ./OC2MRCA-Remediation.sh --action datasource --target C2M-DS # DRY-RUN preview ./OC2MRCA-Remediation.sh --action jms --target C2M-Queue # DRY-RUN preview ./OC2MRCA-Remediation.sh --action restart --target SPLService --apply # change window only Actions: restart | datasource | jms Target: server / data-source / destination name. Admin URL for the emitted WLST steps: ADMIN_URL=t3://your-admin-host:7001 EXPECTED RESULTS DRY-RUN prints the isolated layer, the smallest safe restore for it, and the EXACT command(s) to apply (with the "capture snapshot + backup first" and "re-validate after" reminders) — and exits without changing anything. With --apply it prints the same steps framed as a change-window runbook to execute; it still does not perform a live restart itself. Always re-run OC2MRCA-Validation.py afterwards to confirm the restore. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. Read-only in dry-run. It only ever previews reversible, service-restoration actions; anything that edits config or changes the product is out of scope by design. Command names/flags follow the WebLogic 14c / OUAF model — verify for your version before use.

OC2MRCA-Report.sql

Purpose: Render the database side of the C2M RCA - Diagnostic Snapshot from the C2M schema and the Oracle Database: batch run status and any error/overdue threads, JMS/message- related backlog signals, blocking/blocked sessions (the classic hidden cause of a stall or a stuck JDBC thread), tablespace and temp pressure, and instance uptime. Read-only — SELECT only, it changes no data. Use it to descend into the database layer from the Diagnostic Playbook (Branches B, C, E, F) and to capture the DB baseline. HOW TO RUN Via SQL*Plus or SQLcl against the C2M schema (a read-only account is fine): sqlplus user/pass@your-db @OC2MRCA-Report.sql (or sql user/pass@your-db @OC2MRCA-Report.sql with SQLcl) Some sections read DBA_/V$ views; if the account lacks them those sections note it and the rest still run. EXPECTED RESULTS Sections print in order: (1) instance/uptime, (2) batch runs last 24h with status, (3) errored/overdue batch threads, (4) blocking sessions, (5) tablespace pressure, (6) temp usage, (7) message/backlog signal. PASS = no errored/overdue thread, no blocking session, tablespace/temp within threshold. FAIL = any error/overdue thread, a blocking chain, or a tablespace/temp near full. NOTES White-label, repeatable DB diagnostics — the same query set every incident and for the baseline. Read-only against live systems. Table/column/view names and status-flag values follow the OUAF/C2M and Oracle Database models — verify object names and status codes for your version (they are cited, not invented).

How to run: Via SQL*Plus or SQLcl against the C2M schema (a read-only account is fine): sqlplus user/pass@your-db @OC2MRCA-Report.sql (or sql user/pass@your-db @OC2MRCA-Report.sql with

SQLcl) Some sections read DBA_/V$ views; if the account lacks them those sections note it and the rest still run. EXPECTED RESULTS Sections print in order: (1) instance/uptime, (2) batch runs last 24h with status, (3) errored/overdue batch threads, (4) blocking sessions, (5) tablespace pressure, (6) temp usage, (7) message/backlog signal. PASS = no errored/overdue thread, no blocking session, tablespace/temp within threshold. FAIL = any error/overdue thread, a blocking chain, or a tablespace/temp near full. NOTES White-label, repeatable DB diagnostics — the same query set every incident and for the baseline. Read-only against live systems. Table/column/view names and status-flag values follow the OUAF/C2M and Oracle Database models — verify object names and status codes for your version (they are cited, not invented).

OC2MRCA-ScopeHelper.sh

Purpose: Scope a C2M incident before any check runs: take the observed symptom, print the matching Diagnostic Playbook branch and the layer-isolation ladder for it, and resolve the target set (which tier / Managed Server(s) / node(s)) the other scripts run against — so triage starts bounded ("SPLWeb login on node-a since the 09:15 deploy") instead of "C2M is broken". Read-only — it only reads an optional scope/topology file and prints guidance; it touches no live system. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MRCA-ScopeHelper.sh --symptom wontstart ./OC2MRCA-ScopeHelper.sh --symptom login ./OC2MRCA-ScopeHelper.sh --symptom batch ./OC2MRCA-ScopeHelper.sh --symptom jms ./OC2MRCA-ScopeHelper.sh --symptom iws ./OC2MRCA-ScopeHelper.sh --symptom performance ./OC2MRCA-ScopeHelper.sh --symptom outage Optional topology file (default ./oc2mrca-topology.conf) lines: TIER,SERVER,NODE e.g. web,SPLWeb,node-a EXPECTED RESULTS Prints: the selected branch and its ladder (which layers to check, in order), the scoping questions to answer (when it started / what changed / total vs partial), and — if the topology file exists — the resolved target servers/nodes for the affected tier. Use the output to drive OC2MRCA-Discovery.sh, OC2MRCA-Validation.py, and OC2MRCA-Report.sql against the right target set. NOTES White-label, repeatable scoping — the same symptom map every incident, so triage is bounded and auditable. Read-only. No live-system changes. Tier/server/node names are yours to fill from your topology; branch names match the Diagnostic Playbook (document 3).

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MRCA-ScopeHelper.sh --symptom wontstart ./OC2MRCA-ScopeHelper.sh --symptom login ./OC2MRCA-ScopeHelper.sh --symptom batch ./OC2MRCA-ScopeHelper.sh --symptom jms ./OC2MRCA-ScopeHelper.sh --symptom iws ./OC2MRCA-ScopeHelper.sh --symptom performance ./OC2MRCA-ScopeHelper.sh --symptom outage Optional topology file (default ./oc2mrca-topology.conf) lines: TIER,SERVER,NODE e.g. web,SPLWeb,node-a EXPECTED RESULTS Prints: the selected branch and its ladder (which layers to check, in order), the scoping questions to answer (when it started / what changed / total vs partial), and — if the topology file exists — the resolved target servers/nodes for the affected tier. Use the output to drive OC2MRCA-Discovery.sh, OC2MRCA-Validation.py, and OC2MRCA-Report.sql against the right target set. NOTES White-label, repeatable scoping — the same symptom map every incident, so triage is bounded and auditable. Read-only. No live-system changes. Tier/server/node names are yours to fill from your topology; branch names match the Diagnostic Playbook (document 3).

OC2MRCA-Validation.py

Purpose: Read the live WebLogic 14c / C2M state for each fault-domain layer and print a PASS or FAIL per check, implementing the C2M RCA - Layer Isolation Checklist: Up server states / health (RUNNING vs ADMIN/FAILED) Resourced execute-thread pool health, stuck and hogging thread counts Authenticated

JDBC data-source / connection-pool state (connect, capacity, failures) Resourced JMS destination consumer and pending-message counts (backlog) Authenticated SSL identity/trust certificate expiry Read-only — it only reads runtime MBeans and lists keystore expiry; it changes nothing. Run it during triage to isolate the failing layer, and after a fix to validate the restore (Diagnostic Playbook; Testing Guide Tests 2 and 7). HOW TO RUN java weblogic.WLST OC2MRCA-Validation.py Set the admin URL and read-capable credentials via environment variables first: export ADMIN_URL=t3://your-admin-host:7001 export WLS_USER=your-readonly-admin export WLS_PASS=your-password (Prompted interactively if not set. Keystore expiry uses keytool separately — see NOTES.) EXPECTED RESULTS One PASS/FAIL line per layer check with the reading, then an overall verdict. On a healthy environment every layer reads PASS. Any FAIL names the failing layer to descend into (see the Diagnostic Playbook) and, if a restore is needed, points to OC2MRCA-Remediation.sh (dry-run by default). NOTES White-label, repeatable validation — the same per-layer checks every incident and environment. Read-only against live systems (runtime MBean reads only). MBean paths, health-state names, and status values follow the WebLogic 14c / OUAF model — verify for your version; they are cited, not invented. Certificate expiry is best listed with keytool -list on the identity/trust stores from the domain SSL config (kept separate so this script needs no keystore passwords); this script reports the SSL listener presence and reminds you to check expiry.

How to run: java weblogic.WLST OC2MRCA-Validation.py Set the admin URL and read-capable credentials via environment variables first: export ADMIN_URL=t3://your-admin-host:7001 export WLS_USER=your-readonly-admin export WLS_PASS=your-password (Prompted interactively if not set. Keystore expiry uses keytool separately — see NOTES.) EXPECTED RESULTS One PASS/FAIL line per layer check with the reading, then an overall verdict. On a healthy environment every layer reads PASS. Any FAIL names the failing layer to descend into (see the Diagnostic Playbook) and, if a restore is needed, points to OC2MRCA-Remediation.sh (dry-run by default). NOTES White-label, repeatable validation — the same per-layer checks every incident and environment. Read-only against live systems (runtime MBean reads only). MBean paths, health-state names, and status values follow the WebLogic 14c / OUAF model — verify for your version; they are cited, not invented. Certificate expiry is best listed with keytool -list on the identity/trust stores from the domain SSL config (kept separate so this script needs no keystore passwords); this script reports the SSL listener presence and reminds you to check expiry.